

Mobile Application Development with Flutter

Lecture 30: ListView Widget in Flutter

ListView in Flutter

- ListView is a scrollable list of widgets arranged linearly
- It displays its children sequentially in the scrolling direction, either vertically or horizontally
- There are different types of ListViews:
 1. ListView
 2. ListView.builder
 3. ListView.separated
 4. ListView.custom

Core Types of ListView

Type	Best Use Case	Key Features
ListView	Small, static lists	Simplest to use; renders all children at once
ListView.builder	Large or dynamic lists	Lazily loads only visible items, saving memory
ListView.separated	Lists needing dividers	Similar to .builder but includes a separator between items
ListView.custom	Highly specialized lists	Provides full control over the child model via a SliverChildDelegate

Lazy Loading

- **Lazy loading** is a design pattern that delays the initialization or rendering of an object until the precise moment it is actually needed
- If you have a list of 10,000 items, a standard ListView tries to create all 10,000 widgets immediately, even if the user can only see 5 on their screen
- This crashes apps or causes "jank"
- A **lazy** list only builds the 5 items currently visible (plus a small buffer)
- As you scroll down, it builds the next ones and destroys (or recycles) the ones that move off-screen

Benefits of Lazy Loading

- **Memory Efficiency:** You aren't storing thousands of invisible widgets in RAM
- **Startup Speed:** The list appears instantly because it only has to render a few items to fill the screen
- **Data Usage:** You can pair lazy loading with "Infinite Scrolling" to fetch data from a database only as the user reaches the bottom
- **The Opposite: Eager Loading**
 - The default `ListView(children: [...])` uses **eager loading**. It loads everything "eagerly" upfront
 - This is fine for a settings page with 10 buttons, but terrible for a social media feed

The Anatomy of a Static ListView

- The Default Constructor
- Controlling Flow: ScrollDirection
- Controlling Direction: reverse
- Using Loop Inside ListView

Prepared by Ehtisham Rasheed

Dynamic Builder List

- Ideal for thousands of items or data fetched from an API

```
ListView.builder(  
  itemCount: 20, // Total number of containers  
  itemBuilder: (context, index) {  
    return Container(  
      height: 100,  
      width: 100,  
      color: Colors.blue,  
      margin: const EdgeInsets.all(8.0), // Adds space between them  
    );  
  },  
)
```

- Using Lists with builder

Separated List

- Perfect for adding Dividers between items

```
ListView.separated(  
  itemCount: 20,  
  separatorBuilder: (context, i) =>  
    const Divider(color: Colors.red, height: 30, thickness: 5),  
  itemBuilder: (context, i) => Container(  
    height: 100,  
    color: Colors.blue,  
    child: Center(  
      child: Text('Item $i', style: const TextStyle(color: Colors.white)),  
    ),  
  ),  
)
```


Why Use 'const'

- It is not strictly necessary, but it is a best practice in Flutter
- Here is why we should use it:

1. Performance (The main reason)

- When we use const, Flutter creates that widget only once in memory and reuses it every time the screen updates

2. Avoiding "Jank"

- In a ListView.builder or separated, items are built and destroyed rapidly as you scroll. Using const for static parts helps keep the scrolling smooth and prevents jank

When you CAN'T use 'const'

- We cannot use const if the widget depends on a variable
- **Bad:** `const Text('Item $i')` → This will throw an error because `$i` changes
- **Good:** `Text('Item $i', style: const TextStyle(...))` → The text changes, but the **style** is always the same, so the style can be const.

Custom Lists

- `ListView.custom` is the "low-level" version of `ListView`
- `ListView`. While `.builder` and `.separated` are great for standard lists, `.custom` gives you total control over how items are **created, cached, and disposed**
- It requires a **`SliverChildDelegate`** to manage the items
- Key Differences:
 - **Builder:** Good for 95% of apps. It's easy and handles the delegate for you behind the scenes
 - **Custom:** Good for 5% of apps that need extreme performance optimization or custom logic for how items enter and leave the screen